Hybrid finite element and neural network methods for parametric PDEs

Michel Duprez[1], Emmanuel Franck[2],
Frédérique Lecourtier[1] and Vanessa Lleras[3]

[1]Project-Team MIMESIS, Inria, Strasbourg
[2]Project-Team MACARON, Inria, Strasbourg
[3]IMAG, University of Montpellier, Montpellier

MACS seminar
("Modélisation, Analyse et Calcul Scientifique")

IMAG
INSTITUT MONTPELLIERAIN
ALEXANDER GROTHENDIECK

Context and motivation

Many-query contexts: real-time simulation, digital twins, optimization, uncertainty quantification

$\Rightarrow$ **Goal:** solve a parametric PDE accurately and quickly, for many parameters $\boldsymbol{\mu}$

Context and motivation

Many-query contexts: real-time simulation, digital twins, optimization, uncertainty quantification

$\Rightarrow$ **Goal:** solve a parametric PDE accurately and quickly, for many parameters $\boldsymbol{\mu}$

Classical methods FEM, FV, FD, spectral

Strengths: solid theory, error analysis

Limitations:

- mesh: complex geometries
- cost: many resolutions
- expressivity: fixed spaces

Context and motivation

Many-query contexts: real-time simulation, digital twins, optimization, uncertainty quantification

$\Rightarrow$ **Goal:** solve a parametric PDE accurately and quickly, for many parameters $\boldsymbol{\mu}$

Classical methods FEM, FV, FD, spectral

Strengths: solid theory, error analysis

Limitations:

- mesh: complex geometries
- cost: many resolutions
- expressivity: fixed spaces

Neural networks PINNs, neural operators

Strengths: mesh-free, parametric, universal

Limitations:

- guarantees: none
- training: difficult, sensitive
- single solve: not competitive

Context and motivation

Many-query contexts: real-time simulation, digital twins, optimization, uncertainty quantification

$\Rightarrow$ **Goal:** solve a parametric PDE accurately and quickly, for many parameters $\boldsymbol{\mu}$

Classical methods FEM, FV, FD, spectral

Strengths: solid theory, error analysis

Limitations:

- mesh → complex geometries
- cost → many resolutions
- expressivity → fixed spaces

Neural networks PINNs, neural operators

Strengths: mesh-free, parametric, universal

Limitations:

- guarantees → none
- training → difficult, sensitive
- single solve → not competitive

Our focus: the expressivity of the approximation spaces

Context and motivation

Many-query contexts: real-time simulation, digital twins, optimization, uncertainty quantification

$\Rightarrow$ **Goal:** solve a parametric PDE accurately and quickly, for many parameters $\boldsymbol{\mu}$

Classical methods FEM, FV, FD, spectral

Strengths: solid theory, error analysis

Limitations:

- mesh → complex geometries
- cost → many resolutions
- expressivity → fixed spaces

Neural networks PINNs, neural operators

Strengths: mesh-free, parametric, universal

Limitations:

- guarantees → none
- training → difficult, sensitive
- single solve → not competitive

Our focus: the expressivity of the approximation spaces

Existing enrichments are problem-dependent $\Rightarrow$ two hybridizations: classical structure + neural expressivity

Two hybridizations

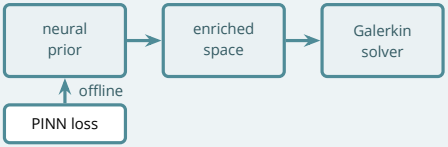
How does the neural network enter the approximation space V_h of a classical method?

Two hybridizations

How does the neural network enter the approximation space V_h of a classical method?

1st hybridization

Enriched FEM using PINNs



Network: pretrained, frozen

Guarantees: a priori error estimates

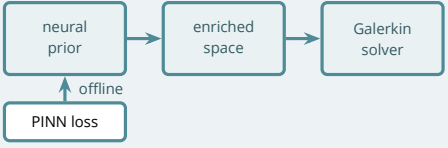
Results: 1D, 2D, 3D, parametric

Two hybridizations

How does the neural network enter the approximation space V_h of a classical method?

1st hybridization

Enriched FEM using PINNs



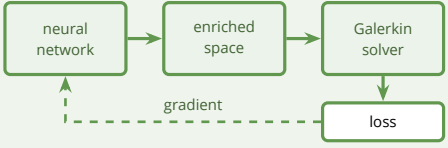
Network: pretrained, frozen

Guarantees: a priori error estimates

Results: 1D, 2D, 3D, parametric

2nd hybridization

Learnable Galerkin methods



Network: learned through the solver

Guarantees: none (experimental)

Results: 1D, 3D, parametric

Parametric elliptic PDEs

Let $\Omega \subset \mathbb{R}^d$ be the spatial domain ($d = 1, 2, 3$) and $\mathcal{M} \subset \mathbb{R}^p$ the parameter space.
For one or several $\boldsymbol{\mu} \in \mathcal{M}$, find $u(\,\cdot\,;\boldsymbol{\mu}) : \Omega \to \mathbb{R}$ such that

$$\begin{cases} \mathcal{L}\big(u\,;\boldsymbol{x},\boldsymbol{\mu}\big) = f(\boldsymbol{x},\boldsymbol{\mu}), & (\boldsymbol{x},\boldsymbol{\mu}) \in \Omega \times \mathcal{M}, \\ \text{Dirichlet, Neumann or Robin BC}, & (\boldsymbol{x},\boldsymbol{\mu}) \in \Gamma \times \mathcal{M}, \end{cases} \tag{$\mathcal{P}$}$$

with $\Gamma := \partial\Omega$ the boundary of Ω, f the source term and $\mathcal{L}$ the linear differential operator defined by

$$\mathcal{L}(\cdot\,;\boldsymbol{x},\boldsymbol{\mu}) : u \mapsto R(\boldsymbol{x},\boldsymbol{\mu})\,u + \boldsymbol{C}(\boldsymbol{x},\boldsymbol{\mu}) \cdot \nabla u - \frac{1}{\mathrm{Pe}} \nabla \cdot \big(D(\boldsymbol{x},\boldsymbol{\mu}) \nabla u\big).$$

where R is the reaction, $\boldsymbol{C}$ the convection, D the (symmetric positive definite) diffusion matrix and Pe the Péclet number.

Numerical methods

Numerical methods for parametric PDEs*

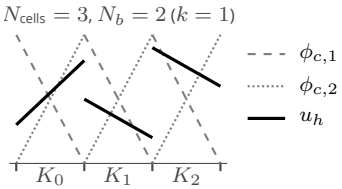
* presented with homogeneous Dirichlet boundary conditions

Discontinuous Galerkin (DG) method[1]

For a fixed $\boldsymbol{\mu} \in \mathcal{M}$, let $\mathcal{T}_h$ be a mesh of Ω of cells $(K_c)_{0 \leq c < N_{\text{cells}}}$.

Let V_h^{disc} be a discontinuous space of piecewise polynomials of degree k on the mesh $\mathcal{T}_h$. Its basis functions $(\phi_{c,i})_{c,i}$, with $\phi_{c,i}$ supported on a single cell K_c, decompose the discrete solution:

$$u_h(\boldsymbol{x}, \boldsymbol{\mu}) = \sum_{c,i} \beta_{c,i}(\boldsymbol{\mu})\, \phi_{c,i}(\boldsymbol{x}).$$



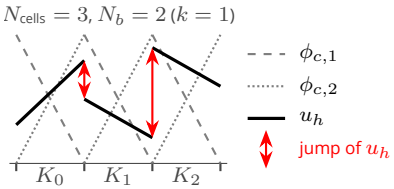
[1] [Arnold et al., 2002]

Discontinuous Galerkin (DG) method[1]

For a fixed $\boldsymbol{\mu} \in \mathcal{M}$, let $\mathcal{T}_h$ be a mesh of Ω of cells $(K_c)_{0 \leq c < N_{\text{cells}}}$.

Let V_h^{disc} be a discontinuous space of piecewise polynomials of degree k on the mesh $\mathcal{T}_h$. Its basis functions $(\phi_{c,i})_{c,i}$, with $\phi_{c,i}$ supported on a single cell K_c, decompose the discrete solution:

$$u_h(\boldsymbol{x}, \boldsymbol{\mu}) = \sum_{c,i} \beta_{c,i}(\boldsymbol{\mu})\, \phi_{c,i}(\boldsymbol{x}).$$



Coupling the cells: ambiguous traces on the interfaces $\Rightarrow$ numerical flux (it defines the method)

Diffusion-dominated regimes: flux for the diffusion term only, here SIPG (Symmetric Interior Penalty Galerkin)

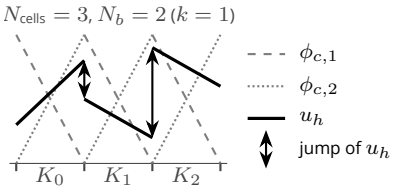
[1] [Arnold et al., 2002]

Discontinuous Galerkin (DG) method[1]

For a fixed $\boldsymbol{\mu} \in \mathcal{M}$, let $\mathcal{T}_h$ be a mesh of Ω of cells $(K_c)_{0 \leq c < N_{\text{cells}}}$.

Let V_h^{disc} be a discontinuous space of piecewise polynomials of degree k on the mesh $\mathcal{T}_h$. Its basis functions $(\phi_{c,i})_{c,i}$, with $\phi_{c,i}$ supported on a single cell K_c, decompose the discrete solution:

$$u_h(\boldsymbol{x}, \boldsymbol{\mu}) = \sum_{c,i} \beta_{c,i}(\boldsymbol{\mu})\, \phi_{c,i}(\boldsymbol{x}).$$



Variational problem (SIPG):

Find $u_h \in V_h^{\text{disc}}$ such that, $\forall v_h \in V_h^{\text{disc}}$, $a_h^{\text{disc}}(u_h, v_h; \boldsymbol{\mu}) = l(v_h; \boldsymbol{\mu}) = \int_\Omega f\, v_h$, with

$$a_h^{\text{disc}}(u_h, v_h; \boldsymbol{\mu}) = \underbrace{\int_\Omega \left(R\, u_h v_h + \boldsymbol{C} \cdot \nabla_h u_h\, v_h + \frac{1}{\text{Pe}} D \nabla_h u_h \cdot \nabla_h v_h \right)}_{\text{cell terms}} \underbrace{- \frac{1}{\text{Pe}} \int_{\mathcal{F}_h} \left([\![u_h]\!] \cdot \{D \nabla_h v_h\} + \{D \nabla_h u_h\} \cdot [\![v_h]\!] \right)}_{\text{consistency}} \underbrace{+ \frac{1}{\text{Pe}} \int_{\mathcal{F}_h} \kappa\, [\![u_h]\!] \cdot [\![v_h]\!]}_{\text{penalty}}$$

where $\mathcal{F}_h$ is the set of faces, ∇_h the broken gradient, $[\![\cdot]\!]$ the jump, $\{\cdot\}$ the average, κ the penalty parameter.

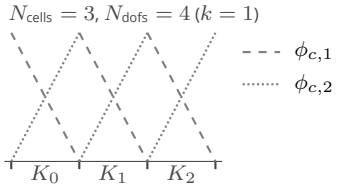
[1] [Arnold et al., 2002]

Continuous Galerkin (CG) method[1]

Same mesh $\mathcal{T}_h$ and fixed $\boldsymbol{\mu}$ as DG.

Let $V_h := V_h^{\text{disc}} \cap C^0(\Omega)$ be a continuous space of piecewise polynomials of degree k on $\mathcal{T}_h$. Its basis functions $(\phi_I)_I$, with ϕ_I supported on the cells sharing the dof I, decompose the discrete solution:

$$u_h(\boldsymbol{x}, \boldsymbol{\mu}) = \sum_{I=1}^{N_{\text{dofs}}} \beta_I(\boldsymbol{\mu})\, \phi_I(\boldsymbol{x}).$$



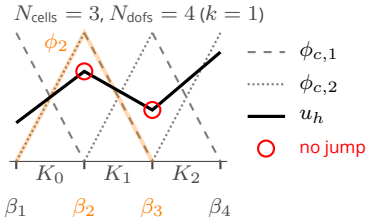
[1] [Ern and Guermond, 2004]

Continuous Galerkin (CG) method[1]

Same mesh $\mathcal{T}_h$ and fixed $\boldsymbol{\mu}$ as DG.

Let $V_h := V_h^{\text{disc}} \cap C^0(\Omega)$ be a continuous space of piecewise polynomials of degree k on $\mathcal{T}_h$. Its basis functions $(\phi_I)_I$, with ϕ_I supported on the cells sharing the dof I, decompose the discrete solution:

$$u_h(\boldsymbol{x}, \boldsymbol{\mu}) = \sum_{I=1}^{N_{\text{dofs}}} \beta_I(\boldsymbol{\mu})\, \phi_I(\boldsymbol{x}).$$



Coupling the cells: shared dofs via the connectivity table $\Rightarrow$ no jumps, no numerical flux

$$\text{conn} : (c, i) \mapsto I, \quad \beta_I(\boldsymbol{\mu}) = \beta_{c,i}(\boldsymbol{\mu}), \quad \phi_I = \sum_{\text{conn}(c,i)=I} \phi_{c,i}$$

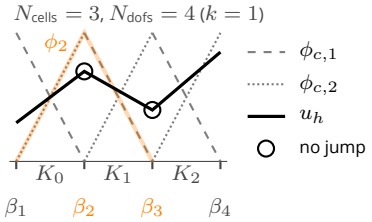
[1] [Ern and Guermond, 2004]

Continuous Galerkin (CG) method[1]

Same mesh $\mathcal{T}_h$ and fixed $\boldsymbol{\mu}$ as DG.

Let $V_h := V_h^{\text{disc}} \cap C^0(\Omega)$ be a continuous space of piecewise polynomials of degree k on $\mathcal{T}_h$. Its basis functions $(\phi_I)_I$, with ϕ_I supported on the cells sharing the dof I, decompose the discrete solution:

$$u_h(\boldsymbol{x}, \boldsymbol{\mu}) = \sum_{I=1}^{N_{\text{dofs}}} \beta_I(\boldsymbol{\mu})\, \phi_I(\boldsymbol{x}).$$



Variational problem: Find $u_h \in V_h^0$ such that, $\forall v_h \in V_h^0$, $a(u_h, v_h; \boldsymbol{\mu}) = l(v_h; \boldsymbol{\mu}) = \int_\Omega f\, v_h$,

$$a(u_h, v_h; \boldsymbol{\mu}) = \boxed{\int_\Omega \left(R\, u_h v_h + \boldsymbol{C} \cdot \nabla u_h\, v_h + \frac{1}{\text{Pe}} D \nabla u_h \cdot \nabla v_h \right)}$$ cell terms

with $V_h^0 \subset V_h$ vanishing on Γ and no interface terms, no penalty.

[1] [Ern and Guermond, 2004]

Galerkin scheme

Linear system (in both methods):

$$\underset{\text{matrix}}{A_h(\boldsymbol{\mu})}\ \underset{\text{dofs}}{\boldsymbol{\beta}_h(\boldsymbol{\mu})} = \underset{\text{right-hand side}}{\boldsymbol{b}_h(\boldsymbol{\mu})}$$

	DG	**CG**
coupling	weak, via numerical fluxes	strong, in the space
$\boldsymbol{\beta}_h(\boldsymbol{\mu})$	$\big(\boldsymbol{\beta}_c(\boldsymbol{\mu})\big)_c$, $\boldsymbol{\beta}_c(\boldsymbol{\mu}) = (\beta_{c,i}(\boldsymbol{\mu}))_i$	$\big(\beta_I(\boldsymbol{\mu})\big)_I$, $I = \mathrm{conn}(c,i)$

Galerkin scheme

Linear system (in both methods):

$$\underset{\text{matrix}}{A_h(\boldsymbol{\mu})}\ \underset{\text{dofs}}{\boldsymbol{\beta}_h(\boldsymbol{\mu})} = \underset{\text{right-hand side}}{\boldsymbol{b}_h(\boldsymbol{\mu})}$$

	DG	**CG**
coupling	weak, via numerical fluxes	strong, in the space
$\boldsymbol{\beta}_h(\boldsymbol{\mu})$	$\big(\boldsymbol{\beta}_c(\boldsymbol{\mu})\big)_c$, $\boldsymbol{\beta}_c(\boldsymbol{\mu}) = (\beta_{c,i}(\boldsymbol{\mu}))_i$	$\big(\beta_I(\boldsymbol{\mu})\big)_I$, $I = \mathrm{conn}(c,i)$
$A_h(\boldsymbol{\mu})$	$A_{(c,i),(c',j)}(\boldsymbol{\mu}) = a_h^{\mathrm{disc}}(\phi_{c',j}, \phi_{c,i}; \boldsymbol{\mu})$	$A_{IJ}(\boldsymbol{\mu}) = a(\phi_J, \phi_I; \boldsymbol{\mu})$
$\boldsymbol{b}_h(\boldsymbol{\mu})$	$b_{(c,i)}(\boldsymbol{\mu}) = l(\phi_{c,i}; \boldsymbol{\mu})$	$b_I(\boldsymbol{\mu}) = l(\phi_I; \boldsymbol{\mu})$

Galerkin scheme

Linear system (in both methods):

$$\underset{\text{matrix}}{A_h(\boldsymbol{\mu})}\ \underset{\text{dofs}}{\boldsymbol{\beta}_h(\boldsymbol{\mu})} = \underset{\text{right-hand side}}{\boldsymbol{b}_h(\boldsymbol{\mu})}$$

	DG	**CG**
coupling	weak, via numerical fluxes	strong, in the space
$\boldsymbol{\beta}_h(\boldsymbol{\mu})$	$\big(\boldsymbol{\beta}_c(\boldsymbol{\mu})\big)_c$, $\boldsymbol{\beta}_c(\boldsymbol{\mu}) = (\beta_{c,i}(\boldsymbol{\mu}))_i$	$\big(\beta_I(\boldsymbol{\mu})\big)_I$, $I = \mathrm{conn}(c,i)$
$A_h(\boldsymbol{\mu})$	$A_{(c,i),(c',j)}(\boldsymbol{\mu}) = a_h^{\mathrm{disc}}(\phi_{c',j}, \phi_{c,i}; \boldsymbol{\mu})$	$A_{IJ}(\boldsymbol{\mu}) = a(\phi_J, \phi_I; \boldsymbol{\mu})$
$\boldsymbol{b}_h(\boldsymbol{\mu})$	$b_{(c,i)}(\boldsymbol{\mu}) = l(\phi_{c,i}; \boldsymbol{\mu})$	$b_I(\boldsymbol{\mu}) = l(\phi_I; \boldsymbol{\mu})$
structure	larger system, one block per cell size: $N_{\mathrm{cells}} \cdot N_b$	smaller system, dofs shared between cells size: N_{dofs}

Physics-informed neural networks (PINNs)[1]

Let $u_\theta(\boldsymbol{x}, \boldsymbol{\mu})$ be a neural network with weights θ and inputs $(\boldsymbol{x}, \boldsymbol{\mu}) \in \Omega \times \mathcal{M}$, approximating the solution u.

Training: Find the weights $\theta^\star$ such that

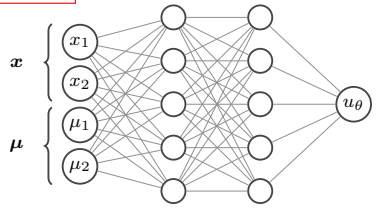
$$\theta^\star = \arg\min_\theta \big(\omega_r \, \mathcal{J}_r(\theta) + \omega_b \, \mathcal{J}_b(\theta)\big), \qquad (\mathcal{P}_\theta)$$

with ω_r and ω_b the weights of the two loss terms, and

residual loss
$$\mathcal{J}_r(\theta) = \int_{\mathcal{M}} \int_{\Omega} \big|\mathcal{L}\big(u_\theta(\boldsymbol{x}, \boldsymbol{\mu}); \boldsymbol{x}, \boldsymbol{\mu}\big) - f(\boldsymbol{x}, \boldsymbol{\mu})\big|^2 \, \mathrm{d}\boldsymbol{x} \, \mathrm{d}\boldsymbol{\mu},$$

boundary loss
$$\mathcal{J}_b(\theta) = \int_{\mathcal{M}} \int_{\Gamma} \big|u_\theta(\boldsymbol{x}, \boldsymbol{\mu})\big|^2 \, \mathrm{d}\boldsymbol{x} \, \mathrm{d}\boldsymbol{\mu}.$$

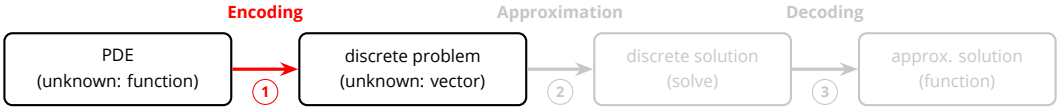
Monte-Carlo method:
Discretize the cost functions by random process.



[1] [Raissi, Perdikaris, and Karniadakis, 2019]

Link between Galerkin and PINNs

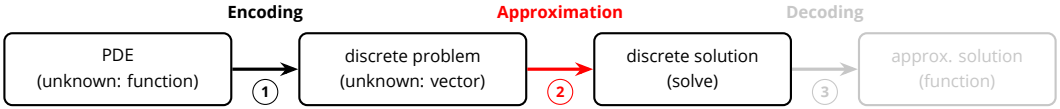
Numerical method (three steps):



		Galerkin (DG/CG)	PINN
	Space	vector space (V_h^{disc}/V_h)	manifold of the u_θ
(1)	Unknowns	dofs $\boldsymbol{\beta}_h(\boldsymbol{\mu})$	weights θ
	Integrals	quadrature on cells	Monte Carlo on collocation points

Link between Galerkin and PINNs

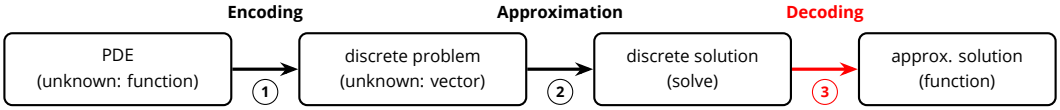
Numerical method (three steps):



		Galerkin (DG/CG)	PINN
①	Space	vector space (V_h^{disc}/V_h)	manifold of the u_θ
	Unknowns	dofs $\boldsymbol{\beta}_h(\boldsymbol{\mu})$	weights θ
	Integrals	quadrature on cells	Monte Carlo on collocation points
②	Parameters	fixed $\boldsymbol{\mu}$, one solve per $\boldsymbol{\mu}$	$\boldsymbol{\mu} \in \mathcal{M}$ as input, one training
	Resolution	linear system $A_h(\boldsymbol{\mu})\boldsymbol{\beta}_h(\boldsymbol{\mu}) = \boldsymbol{b}_h(\boldsymbol{\mu})$	optimization on θ

Link between Galerkin and PINNs

Numerical method (three steps):

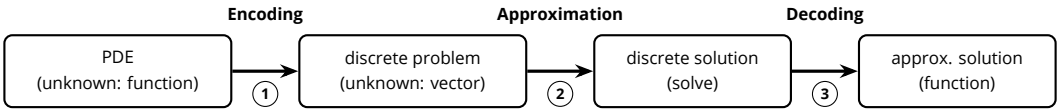


		Galerkin (DG/CG)	PINN
①	Space	vector space (V_h^{disc}/V_h)	manifold of the u_θ
	Unknowns	dofs $\boldsymbol{\beta}_h(\boldsymbol{\mu})$	weights θ
	Integrals	quadrature on cells	Monte Carlo on collocation points
②	Parameters	fixed $\boldsymbol{\mu}$, one solve per $\boldsymbol{\mu}$	$\boldsymbol{\mu} \in \mathcal{M}$ as input, one training
	Resolution	linear system $A_h(\boldsymbol{\mu})\boldsymbol{\beta}_h(\boldsymbol{\mu}) = \boldsymbol{b}_h(\boldsymbol{\mu})$	optimization on θ
③	Solution	$u_h(\boldsymbol{x}, \boldsymbol{\mu})$, linear in $\boldsymbol{\beta}_h$	$u_\theta(\boldsymbol{x}, \boldsymbol{\mu})$, non-linear in θ

Link between Galerkin and PINNs

Numerical method (three steps):

Same philosophy, different approximation space



		Galerkin (DG/CG)	PINN
①	Space	vector space (V_h^{disc}/V_h)	manifold of the u_θ
	Unknowns	dofs $\boldsymbol{\beta}_h(\boldsymbol{\mu})$	weights θ
	Integrals	quadrature on cells	Monte Carlo on collocation points
②	Parameters	fixed $\boldsymbol{\mu}$, one solve per $\boldsymbol{\mu}$	$\boldsymbol{\mu} \in \mathcal{M}$ as input, one training
	Resolution	linear system $A_h(\boldsymbol{\mu})\boldsymbol{\beta}_h(\boldsymbol{\mu}) = \boldsymbol{b}_h(\boldsymbol{\mu})$	optimization on θ
③	Solution	$u_h(\boldsymbol{x}, \boldsymbol{\mu})$, linear in $\boldsymbol{\beta}_h$	$u_\theta(\boldsymbol{x}, \boldsymbol{\mu})$, non-linear in θ

1st hybridization

Enriched finite element method using PINNs

Presentation

Pipeline of the hybrid method

TODO

Additive approach

TODO

Convergence analysis

TODO

Numerical results

Test case 1
Test case 2

Numerical results

Test case 1

Test case 2

Test case 1

TODO

Numerical results

Test case 1

Test case 2

Test case 2

TODO

Conclusion, limitations and perspectives

Conclusion, limitations and perspectives

TODO

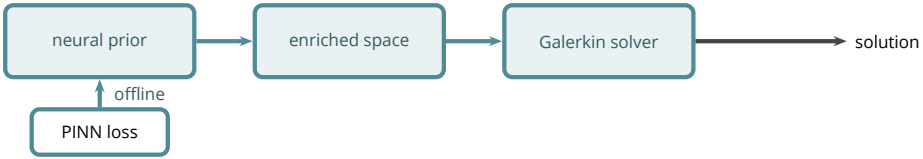
2nd hybridization

Learnable Galerkin methods

Introduction

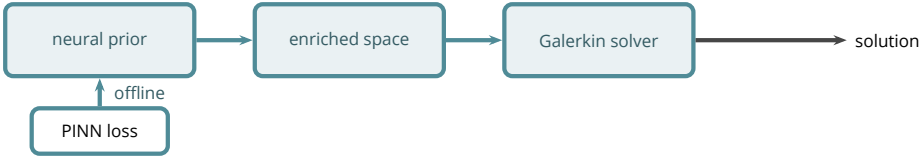
From frozen prior to learned basis

1st hybridization: neural prior trained offline, then frozen: a priori error estimates.
Limit: the gain is bounded by the quality of the prior, trained apart from the scheme.

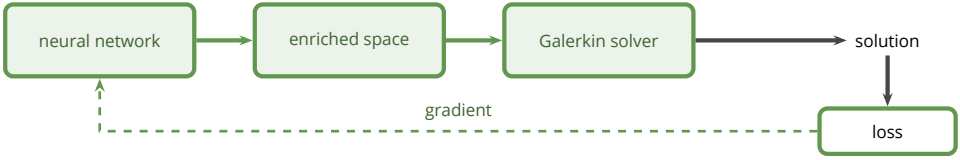


From frozen prior to learned basis

1st hybridization: neural prior trained offline, then frozen: a priori error estimates.
Limit: the gain is bounded by the quality of the prior, trained apart from the scheme.



2nd hybridization: learn the basis through the solver, adapted to the solution of the scheme.
Advantage: basis adapted to the scheme, not bounded by a frozen prior.
Limit: no a priori guarantees, training cost.



Automatic differentiation[1]

Principle: program = graph of elementary operations, chain rule along the graph (JAX[1], ScimBa):
$\mathbf{f} : \mathbb{R}^{n_1} \to \mathbb{R}^{n_2}$, Jacobian $J_{\mathbf{f}}$.

[1] [Bradbury et al., 2018]
[1] [Baydin et al., 2018]

Automatic differentiation[1]

Principle: program = graph of elementary operations, chain rule along the graph (JAX[1], ScimBa):
$\mathbf{f} : \mathbb{R}^{n_1} \to \mathbb{R}^{n_2}$, Jacobian $J_{\mathbf{f}}$.

Forward mode (JVP): $J_{\mathbf{f}}\, \boldsymbol{v}$, efficient for few inputs.

Backward mode (VJP): $\boldsymbol{u}^T J_{\mathbf{f}}$, cost of a few evaluations of $\mathbf{f}$, whatever the number of inputs: backpropagation, used to train N_θ weights.

[1] [Bradbury et al., 2018]
[1] [Baydin et al., 2018]

Automatic differentiation[1]

Principle: program = graph of elementary operations, chain rule along the graph (JAX[1], ScimBa): $\mathbf{f} : \mathbb{R}^{n_1} \to \mathbb{R}^{n_2}$, Jacobian $J_{\mathbf{f}}$.

Forward mode (JVP): $J_{\mathbf{f}}\,\boldsymbol{v}$, efficient for few inputs.

Backward mode (VJP): $\boldsymbol{u}^T J_{\mathbf{f}}$, cost of a few evaluations of $\mathbf{f}$, whatever the number of inputs: backpropagation, used to train N_θ weights.

Through a solver: the output is defined by an equation $\boldsymbol{R}(\boldsymbol{\beta}; \theta) = \mathbf{0}$.

- **Unrolled:** differentiate the iterations, the graph grows with their number.
- **Implicit:** implicit function theorem, no need to differentiate the iterations.

[1] [Bradbury et al., 2018]
[1] [Baydin et al., 2018]

Two settings: PINN and neural operator

Non-parametric PINN loss

parameter	μ fixed
loss	PDE residual
reuse	new μ: new training

Two settings: PINN and neural operator

Non-parametric PINN loss

- parameter: μ fixed
- loss: PDE residual
- reuse: new μ: new training

Parametric neural operator

- parameter: family of μ
- loss: reference solutions
- reuse: unseen μ: no training

Two settings: PINN and neural operator

Non-parametric PINN loss

parameter	$\boldsymbol{\mu}$ fixed
loss	PDE residual
reuse	new $\boldsymbol{\mu}$: new training

Parametric neural operator

parameter	family of $\boldsymbol{\mu}$
loss	reference solutions
reuse	unseen $\boldsymbol{\mu}$: no training

Same learnable Galerkin scheme, different input and loss

Non-parametric enrichment (PINN loss)*

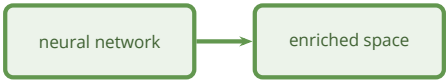
* μ is fixed and omitted from the notation: basis, dofs and solution are valid for this μ only

Neural-enriched Galerkin scheme

neural network

Neural network: Construct a single neural network $v_\theta(\boldsymbol{x})$ on Ω (with N_θ weights), shared by all cells.

Neural-enriched Galerkin scheme



Neural network: Construct a single neural network $v_\theta(\boldsymbol{x})$ on Ω (with N_θ weights), shared by all cells.

Enriched basis: Construct a multiplier $m_\theta(\boldsymbol{x})$ from $v_\theta(\boldsymbol{x})$ and define the enriched basis functions[1] as:

$$\phi^\theta_{c,i}(\boldsymbol{x}) = m_\theta(\boldsymbol{x})\, \phi_{c,i}(\boldsymbol{x}).$$

[1] can be adapted to CG using the connectivity table

Neural-enriched Galerkin scheme



Neural network: Construct a single neural network $v_\theta(\boldsymbol{x})$ on Ω (with N_θ weights), shared by all cells.

Enriched basis: Construct a multiplier $m_\theta(\boldsymbol{x})$ from $v_\theta(\boldsymbol{x})$ and define the enriched basis functions[1] as:

$$\phi^\theta_{c,i}(\boldsymbol{x}) = m_\theta(\boldsymbol{x})\,\phi_{c,i}(\boldsymbol{x}).$$

Galerkin scheme ($\boldsymbol{\mu}$ fixed): Solve the linear system:

$$\boldsymbol{R}_h\big(\boldsymbol{\beta}_h(\theta);\theta\big) = A_h(\theta)\,\boldsymbol{\beta}_h(\theta) - \boldsymbol{b}_h(\theta) = \boldsymbol{0},$$

with $\quad A_{(c,i),(c',j)}(\theta) = a_h^{\text{disc}}(\phi^\theta_{c',j}, \phi^\theta_{c,i}), \qquad b_{(c,i)}(\theta) = l(\phi^\theta_{c,i}).$

[1] can be adapted to CG using the connectivity table

Neural-enriched Galerkin scheme



Neural network: Construct a single neural network $v_\theta(\boldsymbol{x})$ on Ω (with N_θ weights), shared by all cells.

Enriched basis: Construct a multiplier $m_\theta(\boldsymbol{x})$ from $v_\theta(\boldsymbol{x})$ and define the enriched basis functions[1] as:

$$\phi^\theta_{c,i}(\boldsymbol{x}) = m_\theta(\boldsymbol{x})\,\phi_{c,i}(\boldsymbol{x}).$$

Galerkin scheme ($\boldsymbol{\mu}$ fixed): Solve the linear system:

$$\boldsymbol{R}_h\big(\boldsymbol{\beta}_h(\theta);\theta\big) = A_h(\theta)\,\boldsymbol{\beta}_h(\theta) - \boldsymbol{b}_h(\theta) = \boldsymbol{0},$$

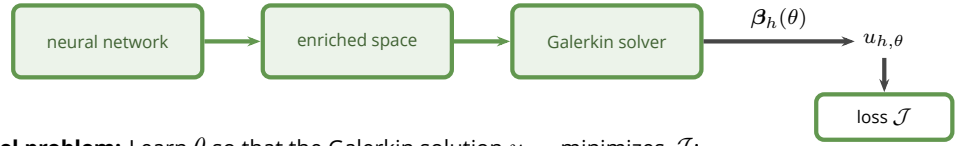
with $\quad A_{(c,i),(c',j)}(\theta) = a_h^{\text{disc}}(\phi^\theta_{c',j}, \phi^\theta_{c,i}), \qquad b_{(c,i)}(\theta) = l(\phi^\theta_{c,i}).$

Solution: Reconstruct the solution in the enriched space as:

$$u_{h,\theta}(\boldsymbol{x}) = \sum_{c,i} \beta_{c,i}(\theta)\,\phi^\theta_{c,i}(\boldsymbol{x}).$$

[1] can be adapted to CG using the connectivity table

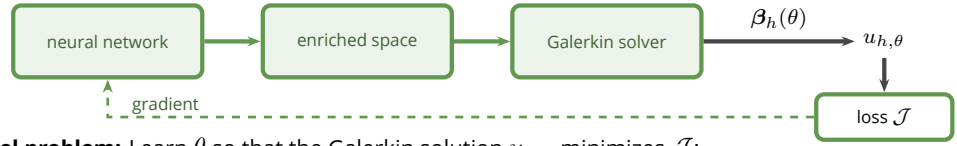
Learning θ: differentiating through the solver



Bi-level problem: Learn θ so that the Galerkin solution $u_{h,\theta}$ minimizes $\mathcal{J}$:

$$\theta^\star = \arg\min_\theta \ \mathcal{J}(\boldsymbol{\beta}_h(\theta);\theta) \quad \text{s.t.} \quad \boldsymbol{R}_h(\boldsymbol{\beta}_h(\theta);\theta) = \mathbf{0}$$

Learning θ: differentiating through the solver



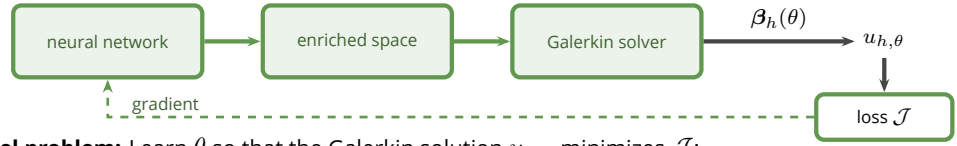
Bi-level problem: Learn θ so that the Galerkin solution $u_{h,\theta}$ minimizes $\mathcal{J}$:

$$\theta^\star = \arg\min_\theta \ \mathcal{J}(\boldsymbol{\beta}_h(\theta);\theta) \quad \text{s.t.} \quad \boldsymbol{R}_h(\boldsymbol{\beta}_h(\theta);\theta) = \mathbf{0}$$

Gradient: Differentiate through the solver (chain rule):

$$\frac{d\mathcal{J}}{d\theta} = \underbrace{\left.\frac{\partial \mathcal{J}}{\partial \theta}\right|_{\boldsymbol{\beta}_h}}_{\text{direct}} + \underbrace{\frac{\partial \mathcal{J}}{\partial \boldsymbol{\beta}_h} \cdot \frac{d\boldsymbol{\beta}_h}{d\theta}}_{\text{through the dofs}}$$

Learning θ: differentiating through the solver



Bi-level problem: Learn θ so that the Galerkin solution $u_{h,\theta}$ minimizes $\mathcal{J}$:

$$\theta^\star = \arg\min_\theta \ \mathcal{J}(\boldsymbol{\beta}_h(\theta);\theta) \quad \text{s.t.} \quad \boldsymbol{R}_h(\boldsymbol{\beta}_h(\theta);\theta) = \mathbf{0}$$

Gradient: Differentiate through the solver (chain rule):

$$\frac{d\mathcal{J}}{d\theta} = \underbrace{\left.\frac{\partial \mathcal{J}}{\partial \theta}\right|_{\boldsymbol{\beta}_h}}_{\text{direct}} + \underbrace{\frac{\partial \mathcal{J}}{\partial \boldsymbol{\beta}_h} \cdot \frac{d\boldsymbol{\beta}_h}{d\theta}}_{\text{through the dofs}}$$

Adjoint: With a gradient-based optimizer, avoid $d\boldsymbol{\beta}_h/d\theta$ (only one linear solve, whatever N_θ):

$$A_h(\theta)^T \boldsymbol{\lambda}_h = \left(\frac{\partial \mathcal{J}}{\partial \boldsymbol{\beta}_h}\right)^T \Rightarrow \frac{d\mathcal{J}}{d\theta} = \left.\frac{\partial \mathcal{J}}{\partial \theta}\right|_{\boldsymbol{\beta}_h} - \boldsymbol{\lambda}_h^T \frac{\partial \boldsymbol{R}_h}{\partial \theta}(\boldsymbol{\beta}_h;\theta)$$

1D test case and PINN loss

Test case: 1D advection-diffusion with a boundary layer, $\Omega = (0,1)$, $\mathrm{Pe} = 20$, $f = 1$:

$$\begin{cases} -\dfrac{1}{\mathrm{Pe}}\,\partial_{xx}u + \boldsymbol{C}\,\partial_x u = f & \text{in } \Omega, \\ u = 0 & \text{on } \Gamma, \end{cases} \qquad \boldsymbol{C}(x,\boldsymbol{\mu}) = \mu_1 + \mu_2 x$$

with $\boldsymbol{\mu} = (\mu_1, \mu_2) = (0.8, 0.8)$ fixed.

1D test case and PINN loss

Test case: 1D advection-diffusion with a boundary layer, $\Omega = (0,1)$, $\mathrm{Pe} = 20$, $f = 1$:

$$\begin{cases} -\dfrac{1}{\mathrm{Pe}}\,\partial_{xx}u + \boldsymbol{C}\,\partial_x u = f & \text{in } \Omega, \\ u = 0 & \text{on } \Gamma, \end{cases} \qquad \boldsymbol{C}(x,\boldsymbol{\mu}) = \mu_1 + \mu_2 x$$

with $\boldsymbol{\mu} = (\mu_1, \mu_2) = (0.8, 0.8)$ fixed.

PINN loss: Minimize residual and boundary terms ($\omega_r = 1$, $\omega_b = 10$):

$$\mathcal{J}(\theta) = \omega_r \underbrace{\int_\Omega \left| -\frac{1}{\mathrm{Pe}}\,\partial_{xx}u_{h,\theta} + (\mu_1 + \mu_2 x)\,\partial_x u_{h,\theta} - f \right|^2 \mathrm{d}\boldsymbol{x}}_{\mathcal{J}_r \text{ (Monte Carlo, redrawn each epoch)}} + \omega_b \underbrace{\int_\Gamma \left| u_{h,\theta} \right|^2 \mathrm{d}\boldsymbol{x}}_{\mathcal{J}_b}$$

1D test case and PINN loss

Test case: 1D advection-diffusion with a boundary layer, $\Omega = (0,1)$, $\mathrm{Pe} = 20$, $f = 1$:

$$\begin{cases} -\dfrac{1}{\mathrm{Pe}}\,\partial_{xx}u + \boldsymbol{C}\,\partial_x u = f & \text{in } \Omega, \\ u = 0 & \text{on } \Gamma, \end{cases} \qquad \boldsymbol{C}(x,\boldsymbol{\mu}) = \mu_1 + \mu_2 x$$

with $\boldsymbol{\mu} = (\mu_1, \mu_2) = (0.8, 0.8)$ fixed.

PINN loss: Minimize residual and boundary terms ($\omega_r = 1$, $\omega_b = 10$):

$$\mathcal{J}(\theta) = \omega_r \underbrace{\int_\Omega \Big| -\frac{1}{\mathrm{Pe}}\,\partial_{xx}u_{h,\theta} + (\mu_1 + \mu_2 x)\,\partial_x u_{h,\theta} - f\Big|^2 \mathrm{d}\boldsymbol{x}}_{\mathcal{J}_r \text{ (Monte Carlo, redrawn each epoch)}} + \omega_b \underbrace{\int_\Gamma \big|u_{h,\theta}\big|^2 \mathrm{d}\boldsymbol{x}}_{\mathcal{J}_b}$$

Validity: Learned for this configuration only:

new $\boldsymbol{\mu}$, k or N_{cells} $\Rightarrow$ new training.

1D test case and PINN loss

Test case: 1D advection-diffusion with a boundary layer, $\Omega = (0,1)$, $\mathrm{Pe} = 20$, $f = 1$:

$$\begin{cases} -\dfrac{1}{\mathrm{Pe}}\,\partial_{xx}u + \boldsymbol{C}\,\partial_x u = f & \text{in } \Omega, \\ u = 0 & \text{on } \Gamma, \end{cases} \qquad \boldsymbol{C}(x,\boldsymbol{\mu}) = \mu_1 + \mu_2 x$$

with $\boldsymbol{\mu} = (\mu_1, \mu_2) = (0.8, 0.8)$ fixed.

PINN loss: Minimize residual and boundary terms ($\omega_r = 1$, $\omega_b = 10$):

$$\mathcal{J}(\theta) = \omega_r \underbrace{\int_\Omega \Big| -\frac{1}{\mathrm{Pe}}\,\partial_{xx}u_{h,\theta} + (\mu_1 + \mu_2 x)\,\partial_x u_{h,\theta} - f\Big|^2 \mathrm{d}\boldsymbol{x}}_{\mathcal{J}_r \text{ (Monte Carlo, redrawn each epoch)}} + \omega_b \underbrace{\int_\Gamma \big|u_{h,\theta}\big|^2 \mathrm{d}\boldsymbol{x}}_{\mathcal{J}_b}$$

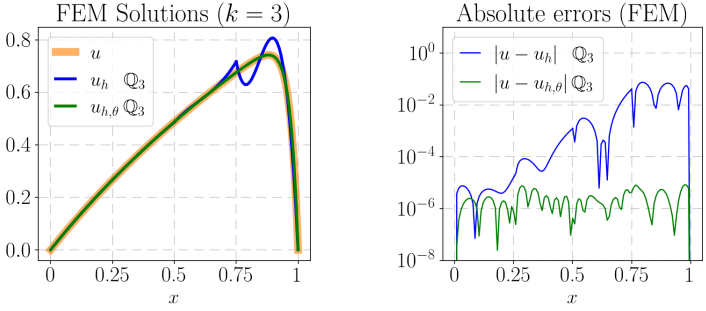
Validity: Learned for this configuration only:

$$\text{new } \boldsymbol{\mu},\ k \text{ or } N_{\text{cells}} \ \Rightarrow\ \text{new training.}$$

Evaluation: Relative L^2 error w.r.t. a reference FEM solution (128 cells, $\mathbb{Q}_4$), never used in the training.

Learned basis: affine multiplier

Setting: CG, $N_{\text{cells}} = 4$ cells, $\mathbb{Q}_3$ ($N_{\text{dofs}} = 13$), affine multiplier $m_\theta = 2 + 0.1\, v_\theta$.
Training: MLP v_θ (tanh, 2×16, $N_\theta = 321$), ENG optimizer[1], 300 epochs, 6000 collocation points.

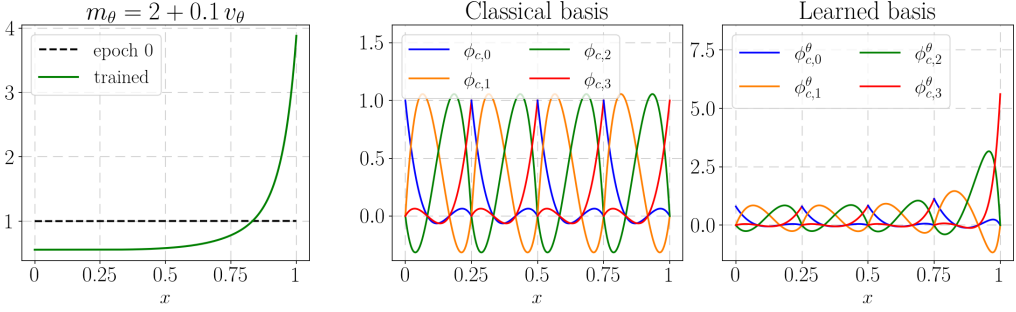


Solutions: Classical u_h oscillates in the boundary layer, $u_{h,\theta}$ matches the reference ($\sim 10^{-5}$ against $7 \cdot 10^{-2}$).

[1] [Müller and Zeinhofer, 2023]

Learned basis: affine multiplier

Setting: CG, $N_{\text{cells}} = 4$ cells, $\mathbb{Q}_3$ ($N_{\text{dofs}} = 13$), affine multiplier $m_\theta = 2 + 0.1\, v_\theta$.
Training: MLP v_θ (tanh, 2×16, $N_\theta = 321$), ENG optimizer[1], 300 epochs, 6000 collocation points.



Solutions: Classical u_h oscillates in the boundary layer, $u_{h,\theta}$ matches the reference ($\sim 10^{-5}$ against $7 \cdot 10^{-2}$).
Basis: The multiplier grows in the boundary layer: the last basis functions are amplified.

[1] [Müller and Zeinhofer, 2023]

Choice of the multiplier

Fix $N_{\text{dofs}} = 25$ and $N_\theta \in \{134, \dots, 151\}$, vary the levers: (N_{cells}, k) and the MLP shape (width, depth).

20 configurations $\times$ 20 seeds $=$ 400 trainings per multiplier (100 epochs, 5000 collocation points).

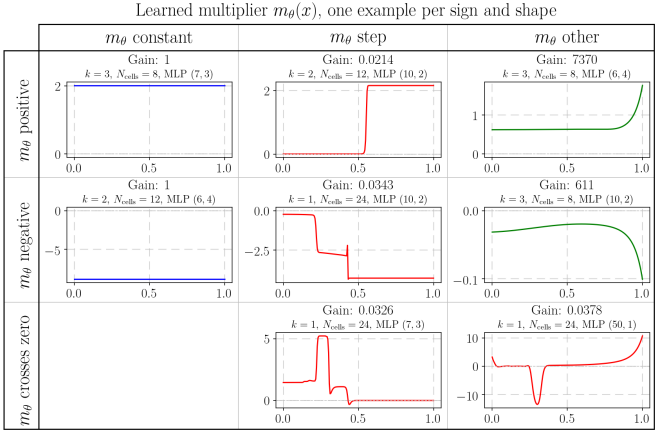
Choice of the multiplier

Fix $N_{\text{dofs}} = 25$ and $N_\theta \in \{134, \dots, 151\}$, vary the levers: (N_{cells}, k) and the MLP shape (width, depth).

20 configurations $\times$ 20 seeds $=$ 400 trainings per multiplier (100 epochs, 5000 collocation points).

Affine multiplier:

$$m_\theta = 2 + 0.1\, v_\theta$$



Choice of the multiplier

Fix $N_{\text{dofs}} = 25$ and $N_\theta \in \{134, \dots, 151\}$, vary the levers: (N_{cells}, k) and the MLP shape (width, depth).

20 configurations $\times$ 20 seeds $=$ 400 trainings per multiplier (100 epochs, 5000 collocation points).

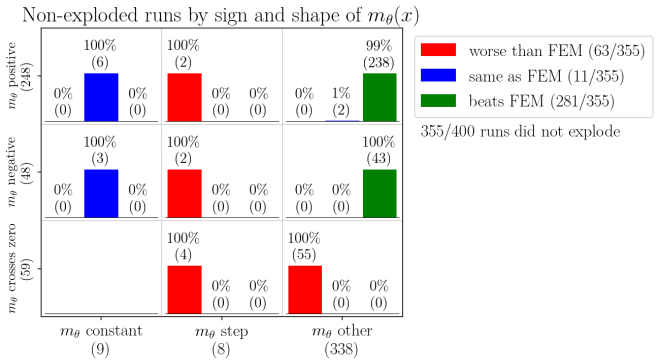
Affine multiplier:

$$m_\theta = 2 + 0.1\, v_\theta$$

Diagnosis:

Some trainings explode (loss $=$ NaN).

Steps and sign changes never beat FEM.



Choice of the multiplier

Fix $N_{\text{dofs}} = 25$ and $N_\theta \in \{134, \dots, 151\}$, vary the levers: (N_{cells}, k) and the MLP shape (width, depth).

20 configurations $\times$ 20 seeds $=$ 400 trainings per multiplier (100 epochs, 5000 collocation points).

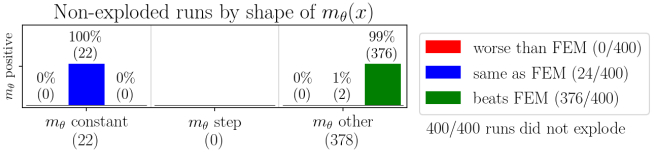
Affine multiplier:

$$m_\theta = 2 + 0.1\, v_\theta$$

Diagnosis:

Some trainings explode (loss $=$ NaN).

Steps and sign changes never beat FEM.



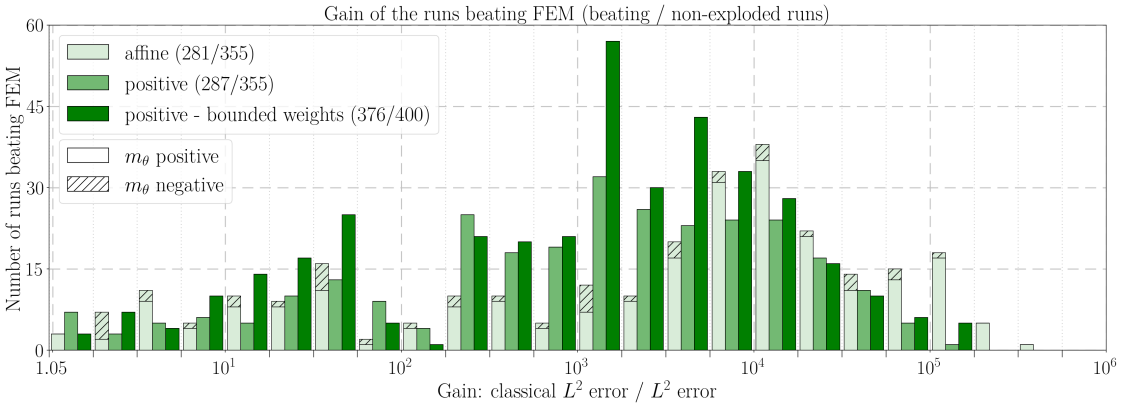
Fix: Positive multiplier

$$m_\theta = \exp(2 \tanh v_\theta)$$

with bounded weights:

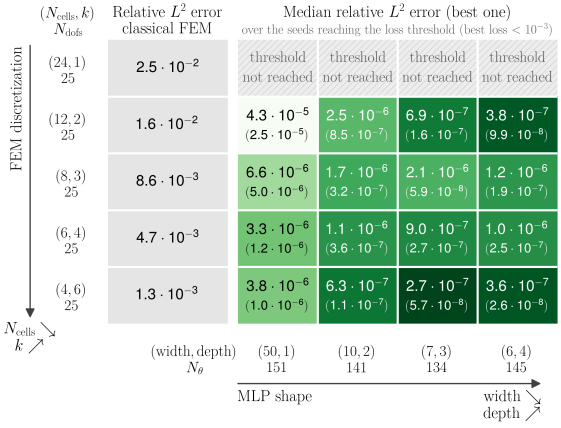
No sign change, no step.

Results at fixed budget



Gain: $376/400$ beat the FEM, none explodes; the affine multiplier keeps the highest gains.

Results at fixed budget



Gain: $376/400$ beat the FEM, none explodes; the affine multiplier keeps the highest gains.

Levers: degree and depth matter: $k = 1$ only a modest gain over FEM, $(50, 1)$ is the worst network.

3D test case

Test case: 3D advection-diffusion with a boundary layer, $\Omega = (0,1)^3$, $\mathrm{Pe} = 20$:

$$\begin{cases} -\dfrac{1}{\mathrm{Pe}}\,\Delta u + \boldsymbol{C}\cdot\nabla u = f & \text{in } \Omega, \\ u = 0 & \text{on } \Gamma, \end{cases} \qquad \boldsymbol{C}(\boldsymbol{x},\boldsymbol{\mu}) = (\mu_1 + \mu_2\,\boldsymbol{x}\cdot\boldsymbol{e}_{\mathrm{diag}})\,\boldsymbol{e}_{\mathrm{diag}},$$

with $\boldsymbol{e}_{\mathrm{diag}} = \frac{1}{\sqrt{3}}(1,1,1)^\top$ the diagonal direction and $\boldsymbol{\mu} = (0.8, 0.8)$ fixed.

3D test case

Test case: 3D advection-diffusion with a boundary layer, $\Omega = (0,1)^3$, $\mathsf{Pe} = 20$:

$$\begin{cases} -\dfrac{1}{\mathsf{Pe}}\,\Delta u + \boldsymbol{C}\cdot\nabla u = f & \text{in } \Omega, \\ u = 0 & \text{on } \Gamma, \end{cases} \qquad \boldsymbol{C}(\boldsymbol{x},\boldsymbol{\mu}) = (\mu_1 + \mu_2\,\boldsymbol{x}\cdot\boldsymbol{e}_{\mathsf{diag}})\,\boldsymbol{e}_{\mathsf{diag}},$$

with $\boldsymbol{e}_{\mathsf{diag}} = \frac{1}{\sqrt{3}}(1,1,1)^\top$ the diagonal direction and $\boldsymbol{\mu} = (0.8, 0.8)$ fixed.

Exact solution: manufactured, vanishing on the 6 faces (f deduced from it):

$$u(\boldsymbol{x}) = \prod_{i=1}^{3} x_i \Big(1 - \exp\Big(-\frac{\mathsf{Pe}\,(\mu_1+\mu_2)}{\sqrt{3}}\,(1-x_i)\Big)\Big)$$

3D test case

Test case: 3D advection-diffusion with a boundary layer, $\Omega = (0,1)^3$, $\mathsf{Pe} = 20$:

$$\begin{cases} -\dfrac{1}{\mathsf{Pe}}\,\Delta u + \boldsymbol{C}\cdot\nabla u = f & \text{in } \Omega, \\ u = 0 & \text{on } \Gamma, \end{cases} \qquad \boldsymbol{C}(\boldsymbol{x},\boldsymbol{\mu}) = (\mu_1 + \mu_2\,\boldsymbol{x}\cdot\boldsymbol{e}_{\mathsf{diag}})\,\boldsymbol{e}_{\mathsf{diag}},$$

with $\boldsymbol{e}_{\mathsf{diag}} = \frac{1}{\sqrt{3}}(1,1,1)^\top$ the diagonal direction and $\boldsymbol{\mu} = (0.8, 0.8)$ fixed.

Exact solution: manufactured, vanishing on the 6 faces (f deduced from it):

$$u(\boldsymbol{x}) = \prod_{i=1}^{3} x_i \Big(1 - \exp\Big(-\frac{\mathsf{Pe}\,(\mu_1+\mu_2)}{\sqrt{3}}\,(1-x_i)\Big)\Big)$$

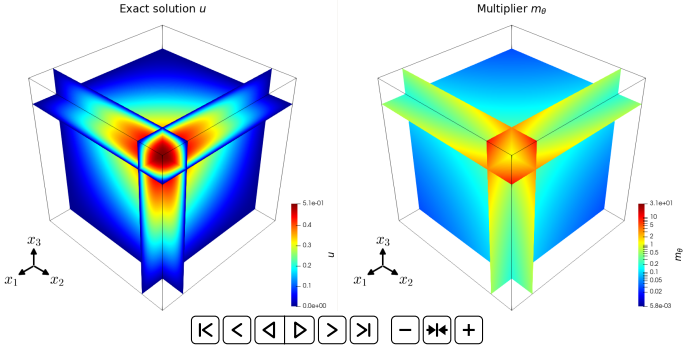
Evaluation: Relative L^2 error w.r.t. the manufactured solution u, never used in the training.

3D: learned multiplier

Setting: CG, $N_{\text{cells}} = 4 \times 4 \times 4$ cells, $\mathbb{Q}_2$ ($N_{\text{dofs}} = 729$), positive multiplier $m_\theta = \exp\left(6 \tanh v_\theta\right)$.
Training: MLP v_θ (tanh, 3×10, $N_\theta = 271$), ENG optimizer, 70 epochs, 6000 collocation points, 3 seeds.

3D: learned multiplier

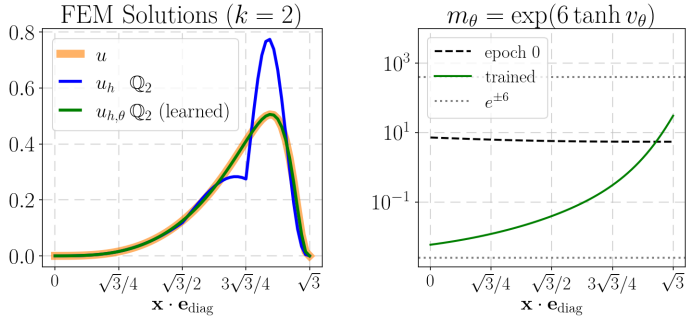
Setting: CG, $N_{\text{cells}} = 4 \times 4 \times 4$ cells, $\mathbb{Q}_2$ ($N_{\text{dofs}} = 729$), positive multiplier $m_\theta = \exp\left(6 \tanh v_\theta\right)$.
Training: MLP v_θ (tanh, 3×10, $N_\theta = 271$), ENG optimizer, 70 epochs, 6000 collocation points, 3 seeds.



Multiplier: grows in the boundary layer, from $6 \cdot 10^{-3}$ to 30 (seed 1).

3D: learned multiplier

Setting: CG, $N_{\text{cells}} = 4 \times 4 \times 4$ cells, $\mathbb{Q}_2$ ($N_{\text{dofs}} = 729$), positive multiplier $m_\theta = \exp\left(6 \tanh v_\theta\right)$.
Training: MLP v_θ (tanh, 3×10, $N_\theta = 271$), ENG optimizer, 70 epochs, 6000 collocation points, 3 seeds.



Multiplier: grows in the boundary layer, from $6 \cdot 10^{-3}$ to 30 (seed 1).
Solution: classical u_h overshoots, $u_{h,\theta}$ matches u.

3D: gain and cost

	time (s)									
	w/ comp.		w/o comp.		N_{dofs}			error		
seed	learn.	class.	learn.	class.	learn.	class.	ratio	learn.	class.	ratio
1	59	51	10.1	49.5	729	$5.31 \cdot 10^5$	729	$5.68 \cdot 10^{-5}$	$3.53 \cdot 10^{-4}$	6.2
2	41	37	6.8	26.6	729	$3.89 \cdot 10^5$	534	$8.13 \cdot 10^{-5}$	$4.82 \cdot 10^{-4}$	5.9
3	41	37	6.9	26.6	729	$3.89 \cdot 10^5$	534	$4.77 \cdot 10^{-4}$	$4.82 \cdot 10^{-4}$	1

Equal time: error $\times 6$ lower for seeds $1, 2$, equal for seed 3.

3D: gain and cost

	time (s)									
	w/ comp.		w/o comp.		N_{dofs}			error		
seed	learn.	class.	learn.	class.	learn.	class.	ratio	learn.	class.	ratio
1	59	51	10.1	49.5	729	$5.31 \cdot 10^5$	729	$5.68 \cdot 10^{-5}$	$3.53 \cdot 10^{-4}$	6.2
2	41	37	6.8	26.6	729	$3.89 \cdot 10^5$	534	$8.13 \cdot 10^{-5}$	$4.82 \cdot 10^{-4}$	5.9
3	41	37	6.9	26.6	729	$3.89 \cdot 10^5$	534	$4.77 \cdot 10^{-4}$	$4.82 \cdot 10^{-4}$	1

Equal time: error $\times 6$ lower for seeds 1, 2, equal for seed 3.

Dofs: 534 to 729 times fewer for the learned basis (without counting the $N_\theta = 271$ weights).

3D: gain and cost

	time (s)									
	w/ comp.		w/o comp.		N_{dofs}			error		
seed	learn.	class.	learn.	class.	learn.	class.	ratio	learn.	class.	ratio
1	59	51	10.1	49.5	729	$5.31 \cdot 10^5$	729	$5.68 \cdot 10^{-5}$	$3.53 \cdot 10^{-4}$	6.2
2	41	37	6.8	26.6	729	$3.89 \cdot 10^5$	534	$8.13 \cdot 10^{-5}$	$4.82 \cdot 10^{-4}$	5.9
3	41	37	6.9	26.6	729	$3.89 \cdot 10^5$	534	$4.77 \cdot 10^{-4}$	$4.82 \cdot 10^{-4}$	1

Equal time: error $\times 6$ lower for seeds 1, 2, equal for seed 3.

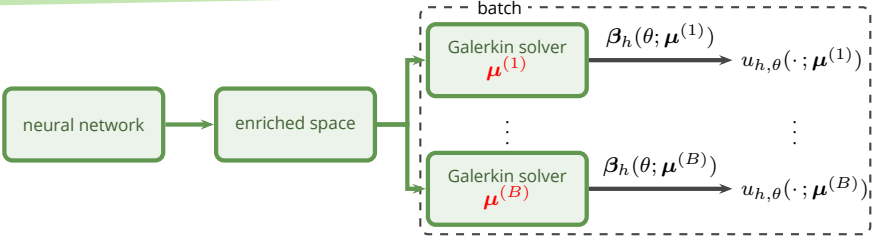
Dofs: 534 to 729 times fewer for the learned basis (without counting the $N_\theta = 271$ weights).

Cost: learned time dominated by compilation: 7 to 10 s without it, against 27 to 50 s for the FEM.

Parametric enrichment (neural operator)*

* $\boldsymbol{\mu}$ is no longer fixed: one network trained once, for all $\boldsymbol{\mu} \in \mathcal{M}$

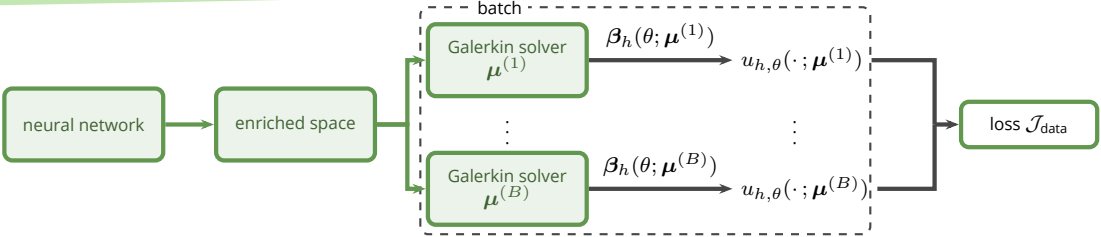
Neural operator: principle



Parameter sampling: $\mathcal{D}_{\text{train}} \subset \mathcal{M}$ chosen once, a random batch of $\boldsymbol{\mu}$ drawn at each epoch.

Parametric scheme: One network, one Galerkin solve per $\boldsymbol{\mu}$ of the batch: $A_h(\theta;\boldsymbol{\mu})\,\boldsymbol{\beta}_h(\theta;\boldsymbol{\mu}) = \boldsymbol{b}_h(\theta;\boldsymbol{\mu})$.

Neural operator: principle



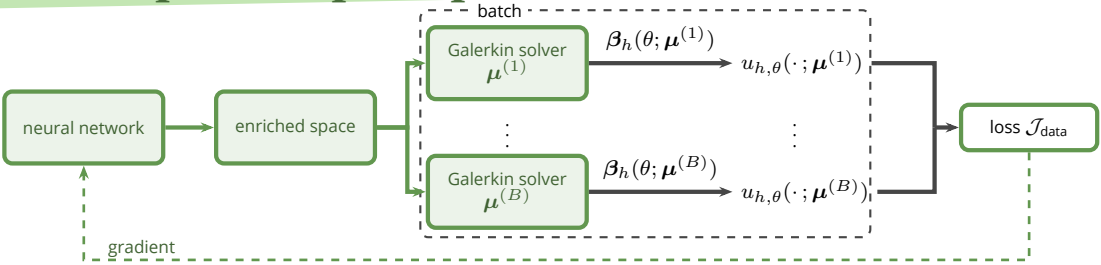
Parameter sampling: $\mathcal{D}_{\text{train}} \subset \mathcal{M}$ chosen once, a random batch of $\boldsymbol{\mu}$ drawn at each epoch.

Parametric scheme: One network, one Galerkin solve per $\boldsymbol{\mu}$ of the batch: $A_h(\theta;\boldsymbol{\mu})\,\boldsymbol{\beta}_h(\theta;\boldsymbol{\mu}) = \boldsymbol{b}_h(\theta;\boldsymbol{\mu})$.

Supervised loss: Fit reference solutions on the batch, at N_x points per $\boldsymbol{\mu}$:

$$\mathcal{J}_{\text{data}}(\theta) = \frac{1}{B\,N_x}\sum_{i=1}^{B}\sum_{j=1}^{N_x}\Big|u_{h,\theta}\big(\boldsymbol{x}^{(j)};\boldsymbol{\mu}^{(i)}\big) - u_{\text{ref}}\big(\boldsymbol{x}^{(j)};\boldsymbol{\mu}^{(i)}\big)\Big|^2.$$

Neural operator: principle



Parameter sampling: $\mathcal{D}_{\text{train}} \subset \mathcal{M}$ chosen once, a random batch of $\boldsymbol{\mu}$ drawn at each epoch.

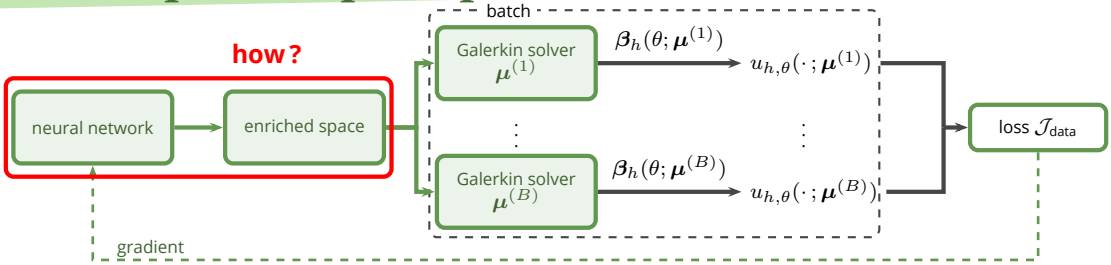
Parametric scheme: One network, one Galerkin solve per $\boldsymbol{\mu}$ of the batch: $A_h(\theta;\boldsymbol{\mu})\,\boldsymbol{\beta}_h(\theta;\boldsymbol{\mu}) = \boldsymbol{b}_h(\theta;\boldsymbol{\mu})$.

Supervised loss: Fit reference solutions on the batch, at N_x points per $\boldsymbol{\mu}$:

$$\mathcal{J}_{\text{data}}(\theta) = \frac{1}{B\,N_x}\sum_{i=1}^{B}\sum_{j=1}^{N_x}\Big|u_{h,\theta}\big(\boldsymbol{x}^{(j)};\boldsymbol{\mu}^{(i)}\big) - u_{\text{ref}}\big(\boldsymbol{x}^{(j)};\boldsymbol{\mu}^{(i)}\big)\Big|^2.$$

Gradient: one adjoint solve per $\boldsymbol{\mu}$ of the batch.

Neural operator: principle



Parameter sampling: $\mathcal{D}_{\text{train}} \subset \mathcal{M}$ chosen once, a random batch of $\boldsymbol{\mu}$ drawn at each epoch.

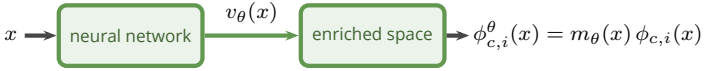
Parametric scheme: One network, one Galerkin solve per $\boldsymbol{\mu}$ of the batch: $A_h(\theta;\boldsymbol{\mu})\,\boldsymbol{\beta}_h(\theta;\boldsymbol{\mu}) = \boldsymbol{b}_h(\theta;\boldsymbol{\mu})$.

Supervised loss: Fit reference solutions on the batch, at N_x points per $\boldsymbol{\mu}$:

$$\mathcal{J}_{\text{data}}(\theta) = \frac{1}{B\,N_x}\sum_{i=1}^{B}\sum_{j=1}^{N_x}\Big|u_{h,\theta}\big(\boldsymbol{x}^{(j)};\boldsymbol{\mu}^{(i)}\big) - u_{\text{ref}}\big(\boldsymbol{x}^{(j)};\boldsymbol{\mu}^{(i)}\big)\Big|^2.$$

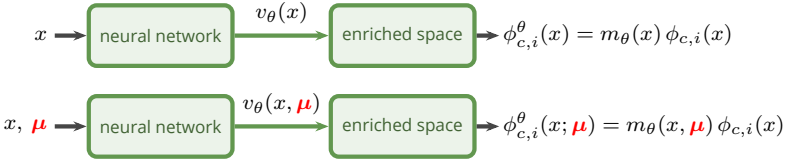
Gradient: one adjoint solve per $\boldsymbol{\mu}$ of the batch.

Constructing the enriched space (three ways)



Shared basis: same enriched space for all μ.

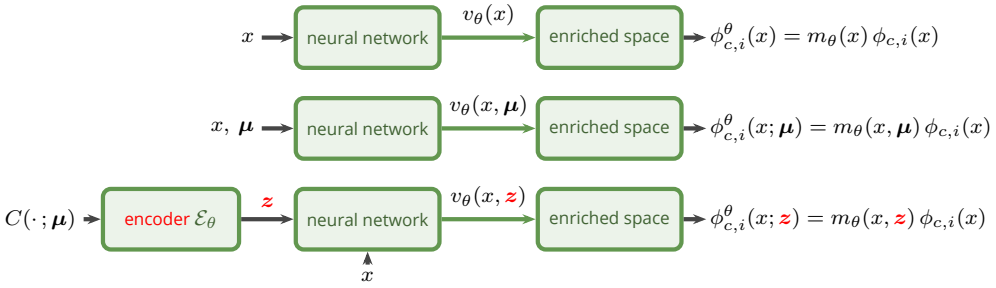
Constructing the enriched space (three ways)



Shared basis: same enriched space for all $\boldsymbol{\mu}$.

$\boldsymbol{\mu}$-dependent basis: one enriched space per $\boldsymbol{\mu}$.

Constructing the enriched space (three ways)



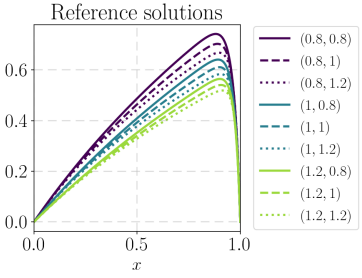
Shared basis: same enriched space for all $\boldsymbol{\mu}$.

$\boldsymbol{\mu}$-dependent basis: one enriched space per $\boldsymbol{\mu}$.

Field-dependent basis: an encoder summarizes the transport field in a latent state $\boldsymbol{z}$.

1D linear transport

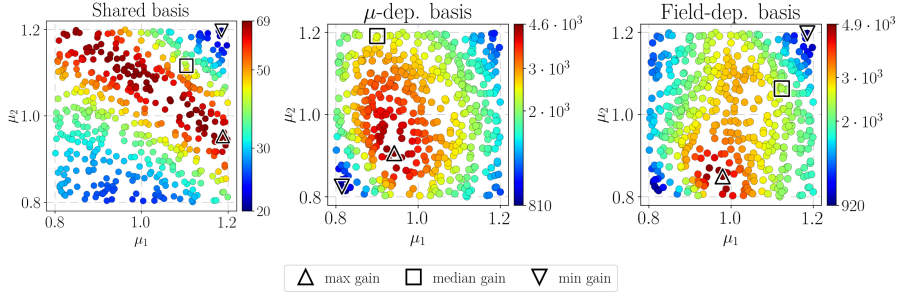
Test case: Same 1D case as the PINN, with $\boldsymbol{C}(x, \boldsymbol{\mu}) = \mu_1 + \mu_2 x$, $\boldsymbol{\mu} \in [0.8, 1.2]^2$: the layer stays at $x = 1$.



1D linear transport

Test case: Same 1D case as the PINN, with $\boldsymbol{C}(x, \boldsymbol{\mu}) = \mu_1 + \mu_2 x$, $\boldsymbol{\mu} \in [0.8, 1.2]^2$: the layer stays at $x = 1$.

Setup: DG, $N_{\text{cells}} = 8$, $k = 2$, 32 training $\boldsymbol{\mu}$, batch of 16, $N_x = 101$ points per $\boldsymbol{\mu}$.

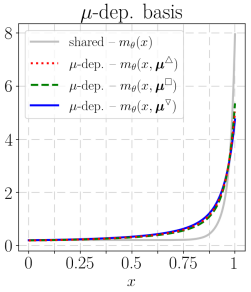


Gain (on 500 unseen $\boldsymbol{\mu}$):
shared 20–69 ($N_\theta = 321$), $\boldsymbol{\mu}$-dep. 811–$4.6 \cdot 10^3$ ($N_\theta = 353$), field-dep. 917–$4.9 \cdot 10^3$ ($N_\theta = 1381$).

1D linear transport

Test case: Same 1D case as the PINN, with $\boldsymbol{C}(x, \boldsymbol{\mu}) = \mu_1 + \mu_2 x$, $\boldsymbol{\mu} \in [0.8, 1.2]^2$: the layer stays at $x = 1$.

Setup: DG, $N_{\text{cells}} = 8$, $k = 2$, 32 training $\boldsymbol{\mu}$, batch of 16, $N_x = 101$ points per $\boldsymbol{\mu}$.

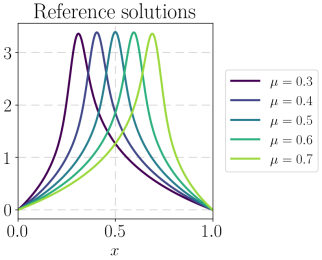


Gain (on 500 unseen $\boldsymbol{\mu}$):
shared 20–69 ($N_\theta = 321$), $\boldsymbol{\mu}$-dep. $811–4.6 \cdot 10^3$ ($N_\theta = 353$), field-dep. $917–4.9 \cdot 10^3$ ($N_\theta = 1381$).

Multipliers: shared, rises only in the last cells; $\boldsymbol{\mu}$-dep., rises over more cells.

1D turning point

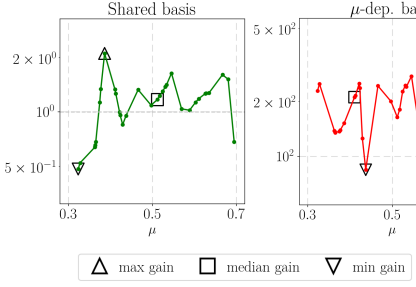
Test case: Same 1D equation as the PINN, $C(x, \mu) = \mu - x$, $\mu \in [0.3, 0.7]$, Pe $= 1000$: layer moves with μ.



1D turning point

Test case: Same 1D equation as the PINN, $\boldsymbol{C}(x, \mu) = \mu - x$, $\mu \in [0.3, 0.7]$, Pe $= 1000$: layer moves with μ.

Setup: Same as the linear transport (DG, $N_{\text{cells}} = 8$, $k = 2$), with a scalar μ.

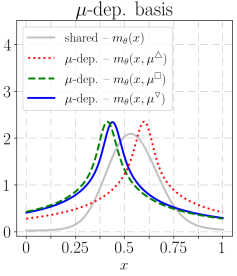


Gain (on 32 unseen μ):
shared 0.5–2.1 $(N_\theta = 321)$, μ-dep. 85–366 $(N_\theta = 337)$.

1D turning point

Test case: Same 1D equation as the PINN, $\boldsymbol{C}(x, \mu) = \mu - x$, $\mu \in [0.3, 0.7]$, Pe $= 1000$: layer moves with μ.

Setup: Same as the linear transport (DG, $N_{\text{cells}} = 8$, $k = 2$), with a scalar μ.



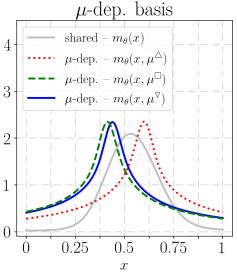
Gain (on 32 unseen μ):
shared 0.5–2.1 $(N_\theta = 321)$, μ-dep. 85–366 $(N_\theta = 337)$.

Multipliers: shared, one central bump; μ-dep., a peak at the layer.

1D turning point

Test case: Same 1D equation as the PINN, $\boldsymbol{C}(x, \mu) = \mu - x$, $\mu \in [0.3, 0.7]$, Pe $= 1000$: layer moves with μ.

Setup: Same as the linear transport (DG, $N_{\text{cells}} = 8$, $k = 2$), with a scalar μ.



Gain (on 32 unseen μ):
shared 0.5–2.1 $(N_\theta = 321)$, μ-dep. 85–366 $(N_\theta = 337)$.

Multipliers: shared, one central bump; μ-dep., a peak at the layer.

Conclusion, limitations and perspectives

Conclusion, limitations and perspectives

TODO

Conclusion

Conclusion

Contribution 1 : TODO

TODO

Contribution 2 : TODO

TODO

Perspectives :

- TODO

Appendix – 1st hybridization

Enriched finite element method using PINNs

Additional details

Additional details

TODO

Détails de la méthode (hors présentation principale)

Additional numerical results

Additional numerical results

TODO

Résultats numériques supplémentaires

Appendix – 2nd hybridization

Learnable Galerkin methods

Additional details

Additional details

TODO

Détails de la méthode (hors présentation principale)

Additional numerical results

Additional numerical results

TODO

Résultats numériques supplémentaires